From netlist to verified model

Automatic real-number-model and testbench generation

for mixed-signal verification

Veylon Systems

Abstract

A real-number model is a fast stand-in for an analog block. SPICE solves the physics of every transistor. A real-number model instead computes an output number from an input number with ordinary arithmetic. That lets it run in a normal digital simulator, next to the chip's logic, fast enough for a full regression suite.

The trouble is that someone has to write the model, keep it matching the circuit as the circuit changes, and show the team that it still matches. The first two jobs are slow. The third usually never happens.

spice2rnm does all three with one command. You give it a SPICE netlist. It gives you back a SystemVerilog real-number model, a UVM-MS verification environment that tests the model, and evidence that both agree with the real circuit.

Three rules make that evidence trustworthy. First, every expected value in every generated testbench is a *SPICE measurement of your circuit*, never a value computed from the model's own equations, so a test cannot pass just by agreeing with itself. Second, every generated environment is shown to be able to fail: the tool deliberately breaks the model and confirms that the test then fails. Third, each block is checked on what it is for. A filter is checked on its frequency response, a duty-cycle corrector on where its clock edges land, and a phase interpolator on how far it shifts the clock at each setting.

The paper works through four circuits, from simple to hard. It runs the last one, a PLL, twice. The first run is an SV–SPICE co-simulation: the digital RTL runs in the SystemVerilog simulator while the analog blocks run as the transistor-level circuit in SPICE, joined by a bridge. This checks the generated models against the real circuit. The second run is SystemVerilog only: the RTL and the generated models, with no SPICE at all. Every number in the paper was produced by the shipped tool, in about an hour on a laptop.

On one page

What you give it. A SPICE netlist, either flat or with subcircuits. Optionally, also a plain-language specification and an existing model library whose conventions the tool should follow. You do not need to label ports, supply a model template or write a testbench.

What you get back, from one command. Three things:

- a SystemVerilog real-number model, whose structure the tool chooses from measurements of the circuit;
- a UVM-MS environment that tests the model, in which every expected value is a SPICE measurement of your circuit;
- evidence that the model matches the circuit. The circuit and the model get identical inputs, their outputs are compared in the way that matters for that block, and the waveforms are saved.

Results on five circuits. Each is one command. All five together took about an hour on a laptop, using open-source simulators.

Circuit	What is compared	Worst vs pass/fail limit	Environment
two-pole filter	waveform, unseen input	0.0097 vs 0.15	64 checks, 0 failed
duty-cycle corrector	where its edges land	0.428 vs 1.0 pp	60 checks, 0 failed
phase interpolator	edge position per code	1.83 vs 6.25 ps	9 checks, 0 failed
PLL, SV–SPICE co-sim	system vs SPICE circuit	3.851 mV, 41% of limit	5 blocks, 0 failed
PLL, SV only	loop closed in SV alone	locked, −0.0005%	12 checks, 0 failed

Why a passing result here is worth believing. Three rules hold throughout.

- The expected answers come from SPICE, never from the model. A test cannot pass by agreeing with itself.
- Every environment is shown to be able to fail. The tool deliberately breaks the model, confirms the test fails, and measures the smallest break the test still catches.
- Each block is checked on its own job: a filter on its frequency response, a clock buffer on its edges, an oscillator on its frequency. A check that does not apply to a block is reported as "not applicable", never counted as a pass.

Where the evidence is. Every model, environment and pass/fail result in this paper is published exactly as the tool wrote it, at `https://github.com/vrajeshprakhya/spice2rnm-demos`. One command regenerates all of it. The outputs are plain IEEE 1800 SystemVerilog with no vendor dependence. Here they ran on ngspice and on the open-source simulator xezim; they will also run on the simulators your flow already uses.

Contents

Terms used in this paper 3

1 The problem 4

2 What you get from one command 5

3 Case study 1: two-pole filter 6

4 Case study 2: duty-cycle corrector 7
4.1 A model that follows the circuit's dynamics . . . 8
4.2 Verified by function, not by formula . . . 8

5 Case study 3: Phase interpolator 9

6 Case study 4: Phase-locked loop 10
6.1 Checked as a system, against the transistor-level circuit . . . 11
6.2 The supply noise the netlist declares is the noise the model sees . . . 12
6.3 Where the block works is measured, not assumed . . . 13
6.4 The same loop, in SystemVerilog alone . . . 14

7 Results at a glance 20
7.1 What the equivalence folders hold . . . 22

8 What the equivalence check compares 23
8.1 What keeps a figure from flattering itself . . . 24
8.2 Blocks, and then the system . . . 24
8.3 One thing a score at an operating point cannot see . . . 25

9 Why a green testbench is worth believing 27

10 Functional specifications as input 28

11 Joining an existing model library 28

12 AI where it helps, physics where it decides 29

13 Scope and requirements 30

A What the routes measure 32
A.1 Why an oscillator is compared as a rate . . . 32
A.2 The tuning curve . . . 33
A.3 Control states . . . 33
A.4 The state space . . . 34

Terms used in this paper

Real-number model (RNM) A SystemVerilog model of an analog block. It computes output values from input values with ordinary arithmetic, so it runs in a digital simulator.

Netlist The SPICE description of a circuit: its components and how they connect. (Older SPICE usage calls it a *deck*; the tool's own generated files still use that word.)

The transistors Short for the transistor-level circuit in SPICE (here, ngspice): the real circuit, simulated in full. Every model is compared against it.

SV–SPICE co-simulation One simulation in which the design's digital RTL runs in the SystemVerilog simulator while its analog part runs as the transistor-level circuit in SPICE. A bridge passes signals between the two. In this paper, "co-simulation" always means this.

SV-only system simulation Also "SystemVerilog only". The design's RTL and the generated models of its analog blocks all run in the SystemVerilog simulator. No SPICE is involved.

Generated Written as output by spice2rnm. A file "not generated" is one the tool deliberately does not write.

Equivalence check Giving the circuit (in SPICE) and the model the same input and comparing their outputs.

Golden value An expected value in a testbench. It is measured from the circuit in SPICE, never taken from the model.

Route The characterisation method the tool picks for a block: for example, measuring a filter's frequency response, or an oscillator's frequency against its control voltage (its tuning curve).

Bar Short for the pass/fail limit a result is compared against.

Verdict The outcome of a check: pass or fail.

Reported for information A number that is shown for information, with no pass/fail limit applied to it.

rms Root mean square: the standard measure of the size of a noise signal or an error.

pp Percentage points (for example, of duty cycle).

Random sample, fixed random seed A simulation with noise gives a different result each run, so one run's result is one random sample of the possible results. Fixing the random seed makes the random number generator repeat, so a rerun gives the same result.

White and flicker noise White noise has equal power at all frequencies. Flicker (or 1/f) noise gets stronger towards low frequencies.

Jitter Variation in when clock edges arrive. *Period jitter* is the variation in the time between one edge and the next.

Composed model The individual block models wired together into a model of the whole circuit.

UVM-MS Accellera's Universal Verification Methodology for mixed-signal designs, built on IEEE 1800 SystemVerilog.

`wreal`, nettype Two ways of declaring nets that carry analog (real) values in SystemVerilog. `wreal` comes from Verilog-AMS; a user-defined nettype is the IEEE 1800 equivalent.

`MANIFEST.txt` The list of large output files that are not stored in the repository. For each file it gives the size and the command that recreates it. Such a file is "listed in `MANIFEST.txt`".

1 The problem

A model of an analog block must be both fast and accurate, and the two goals pull against each other. It must be fast, or it cannot run in a digital regression suite at all. It must be accurate, or passing its tests means nothing.

Hand-written models drift away from the circuits they represent, and nothing signals the drift. The model still compiles, still simulates and still passes its tests. It has simply stopped describing the circuit.

The usual safeguard is a hand-written testbench, but it has the same problem one level up. A person decides what to check and how close counts as close enough. Those decisions are rarely recorded next to the numbers they produced. When the circuit changes, the model, the testbench and the tolerances all need revisiting by hand, and in practice that does not happen.

spice2rnm regenerates all three from the netlist whenever you ask. The model, the environment that checks it and the evidence all come from the same command. So they cannot quietly fall out of step with the circuit or with each other.

2 What you get from one command

```
spice2rnm dcc.cir --emit-uvm-ms --spec dcc_spec.md
```

You do not name the ports: the tool finds them in the netlist. The supplies carry only a DC level. The input is the node driven by the one source that carries a transient waveform or an AC magnitude. On this netlist, that node is `clk`.

Finding the output is a different problem, because the wiring alone does not say which node you care about. The netlist's own `.save`, `.print` or `.probe` statement does say it: that is the author naming the signal the netlist exists to show. When a netlist has one, the tool takes the output from it. When it has none, as here, the tool guesses the output from the node names. The run reports which of the two it did, so you can tell a declared output from a guessed one.

Four routes (a route is the characterisation method the tool picks for a block) never guess. These are the ones that measure the edge, frequency or current of one particular node. For them, you must name the output.

The command produces three things.

- **A SystemVerilog real-number model.** The tool chooses the model's structure to match the block's measured behaviour. This includes blocks whose dynamics change with signal level, where any fixed-response model is wrong over most of its range.

  The model has `wreal` ports, so it connects directly into an RNM library like the models next to it. The model itself connects; there is no wrapper around it.

  This choice affects function, not just style. A `real` port is a *variable*, and a variable accepts exactly one driver. Nothing else can share the node, and a bridge cannot attach without a converter module in between. A `wreal` port is a *net*, so several drivers can combine on it: an offset source, a noise injector or a second bridge. The model then drops into a library with nothing between it and the nets it is wired to.

  There is one real cost. `wreal` is Verilog-AMS, not IEEE 1800, so the model compiles only on simulators that support it, and everything that instantiates the model inherits that need. For a flow without `wreal`, `--model-ports real` generates the same model with variable ports. It depends on no package, no nettype and no language extension. `--model-ports house` instead gives the model your own library's nettype (Section 11).

  The computation is identical in every case; only the port declarations differ. One detail: a net cannot be assigned inside procedural code, so a model with net ports drives its output through a single continuous assignment. We measured that the choice makes no difference: the filter scores `rms_error_norm` 0.029008790476737805 against SPICE with either port type, identical to every digit.

- **A UVM-MS verification environment** that follows the Accellera UVM-MS specification. It is built on IEEE 1800, not on Verilog-AMS.

  Analog values travel on *user-defined nettypes* (IEEE 1800-2017 §6.6.7). The generated package declares the nettype together with a generated resolution function, `sum_res`, that adds up the contributions of all drivers. So when several drivers share a net (an offset source, a noise injector, a second bridge), the nettype combines them, with no hand-edited testbench code.

  The environment instantiates the model itself, not a copy behind a wrapper, so the checks cover exactly what ships. The environment's own structure is IEEE 1800 throughout: the nettype, the resolution function and the `interconnect` ports. The model it drives has `wreal` ports, though, so running it needs a simulator that supports `wreal`.

  Bridge and wrapper ports are `interconnect` (§6.6.8), so they take on the testbench's nettype instead of naming one. The scoreboard's golden values are SPICE measurements of your circuit.

- **Equivalence evidence.** The circuit and the model get identical inputs. Their outputs are then compared in the way that matters for the block: waveform error for blocks on the signal path, edge position for blocks on the clock path. The aligned waveforms are saved, so any pass/fail result can be recomputed from its own run directory.

An optional plain-language functional specification supplies what no netlist can: the block's intended operating range and its acceptance criteria (Section 10).

3 Case study 1: two-pole filter

The first case is a two-pole RC low-pass filter. It is an ordinary block: it takes a signal in and shapes its frequency response on the way out. Below is the whole input netlist, apart from its comment lines:

```
2-pole passive RC low-pass filter -- fast-running RNM test case
VIN vin 0 DC 0.9 AC 1
R1  vin  mid  1k
C1  mid  0    15.9p
R2  mid  vout 1k
C2  vout 0    15.9p
.end
```

That is all the tool is given. There are no directives naming the ports, no hint about what the block is for, and no declared corner frequency. The generated model is then checked in four independent ways:

Chirp (swept-tone) equivalence	0.0290 against a 0.15 threshold	PASS
PRBS (independent stimulus)	0.0097 normalised	PASS
UVM-MS, DC section	49 checks, 0 failed	
UVM-MS, AC section	15 checks, 0 failed	
worst magnitude	0.0674 dB of a 0.25 dB tolerance	
worst phase	2.70° of an 8.00° tolerance	

Below is the head of the generated file, `rc_lpf2_rnm.sv`, exactly as published:

```
// SystemVerilog Real Number Model, auto-generated by spice2rnm.
// Fitted transfer function: H(s) = 1 * 1  / ((1+s/2.4023e+07) * (1+s/1.64656e
    +08))
```

```
// poles (Hz): [3823382.3097665044, 26205852.219306976]
// fit quality: rms_error_db=1.417e-08  max_error_db=5.81e-08

module rc_lpf2_rnm(input wreal in_val, output wreal out_val);
```

The poles were measured from the circuit, not declared by anyone. The file also records how closely its fitted transfer function matches those measurements: the `fit quality` line gives the rms (root mean square) and worst-case error in dB. A reader can judge the model's accuracy without running anything.

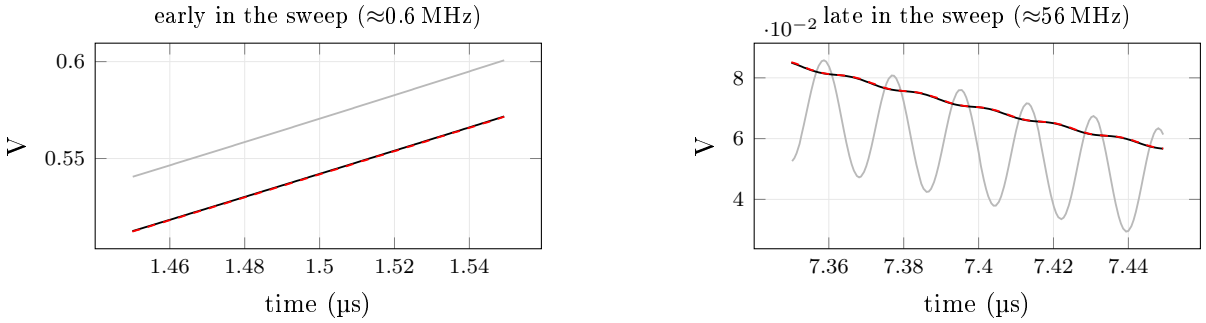
The model also states where its results stop counting as evidence. Every model carries the input range it was characterised over. If an input falls outside that range, the model reports it instead of quietly returning a value:

```
localparam real DOMAIN_LO = 0.0;
localparam real DOMAIN_HI = 1.8;

a_input_domain: assert (in_val >= DOMAIN_LO - DOMAIN_EPS
                     && in_val <= DOMAIN_HI + DOMAIN_EPS)
else ... $warning("%m: input %0.6g V is outside the characterized
  range %0.6g..%0.6g V. static_curve clamps there, so this model
  keeps running and returns the endpoint value -- it is not
  extrapolating, and it is no longer evidence. Reported once.");
```

Asked for a voltage it was never characterised at, the model neither fails nor extrapolates. It clamps: it keeps returning the value at the edge of its range. That value looks plausible, which is what makes this case dangerous and why the warning exists. The warning fires once per model, so the log of a long run stays readable.

The PRBS row matters more than it looks. The model is built from a chirp, a tone whose frequency sweeps upward. It is then tested with a pseudo-random bit sequence (PRBS) it has never seen. A model that had only memorised the stimulus it was built from would fail here. This one does not.



Grey is the stimulus, black the transistor-level circuit in SPICE, red dashed the generated model. The two panels are two moments of the same chirp, at the simulator's own time resolution. Early in the sweep, the output follows the input. Late in the sweep, the input still swings but the output barely moves: that is the filter at work. Measured over 100 ns windows, the output-to-input swing ratio is 0.99 in the first window and 0.50 in the second, and the model tracks the transistor-level circuit through both. Data taken from `waveform_compare.csv` and the PWL (piecewise-linear) stimulus in that run's own `equivalence/` directory.

4 Case study 2: duty-cycle corrector

A clock is meant to be high for half of each cycle. The fraction of each cycle it spends high is its duty cycle, and a duty-cycle corrector restores it when it is wrong. This one is built around a comparator.

The block's speed has two different answers, depending on how you measure it. Nudge the input slightly (a small-signal measurement) and watch the output move: the answer is 3.7×10^8 rad/s. Swing the input hard and time the output edges directly: the answer is between 1.3×10^{10} and 5.1×10^{10} rad/s, which is 35 to 137 times faster.

The slow answer is not wrong. It measures a sensitive node inside the comparator, and that node responds slowly. But the output is not taken from that node. It comes from a buffer that switches hard between the supply rails without waiting for it. So the slow number is true of a node the output does not follow, and a model built on it runs tens to hundreds of times too slow.

The fast answer is not a single number either. It changes by a factor of 3.9 across the input range, so a model with one fixed speed cannot follow it.

This sets a trap for the usual check. That check compares the model's AC (small-signal frequency) response with the circuit's. Both are measured by nudging, so both report the slow node, and the two agree. The check passes a model that is a hundred times too slow and never reveals the error.

That is why hand-written models of blocks like this tend to be the ones people trust least.

4.1 A model that follows the circuit's dynamics

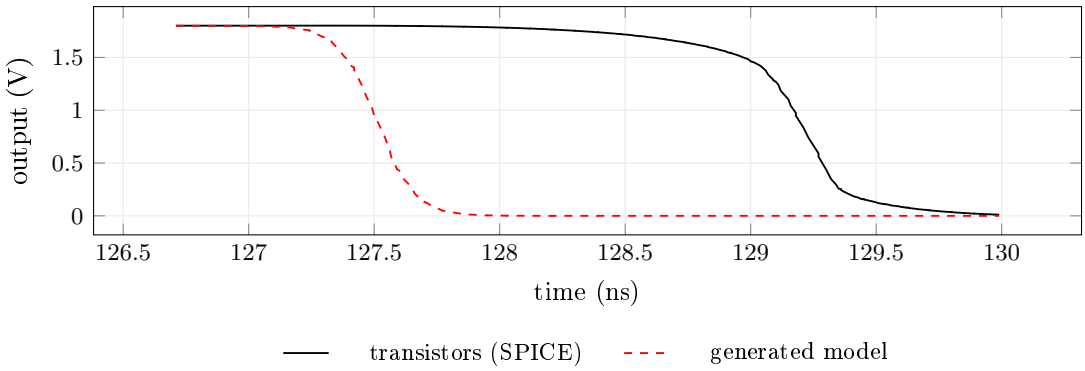
While measuring the circuit, spice2rnm detects that its speed depends on input level. It then builds a model whose speed varies with level as the measurements say it should. The model also reproduces the circuit's rise/fall asymmetry: the circuit takes a measured 95 ps longer to switch one way than the other. Fed a clock that is high exactly half the time, the model's output duty matches the circuit's to 0.000 percentage points (pp).

Transient equivalence	0.0769 against a 0.15 threshold	PASS
UVM-MS, DC section	46 checks, 0 failed	
UVM-MS, duty section	14 checks, 0 failed	
worst duty error	0.428 pp of the specification's 1.0 pp	

4.2 Verified by function, not by formula

The UVM-MS environment generated for this block has *no AC section*. That is a deliberate decision based on measurement, not an oversight. An AC sweep assumes the circuit sits at one operating point and moves only gently around it. A comparator in normal use never does that: it switches hard between two states. Checked that way, this model fails every single point by more than 55 dB, even though it is demonstrably accurate. That failure says nothing about the model and everything about using the wrong check.

So the generated environment checks what a duty-cycle corrector is actually for: the duty of its output. The tool drives the transistor-level circuit in SPICE with clocks at two rates and three input duties. These are the rates and the 35–65% duty range that the block's specification states (Section 10). Each time, the tool records the output duty SPICE produces (the golden value), and the model must reproduce those recorded values. The recorded output duties span 30 percentage points, wide enough that a model ignoring its input could not pass by luck.



One falling edge, at the simulator's own time resolution. The model places this edge 1.68 ns early and the next edge 1.60 ns early, so the interval between them (the duty this block is specified on) is preserved to 0.03%. An average voltage error over the whole run would report this as a large disagreement. The duty check that the testbench actually applies does not.

5 Case study 3: Phase interpolator

A phase interpolator does not change a signal's amplitude. It shifts the signal in *time*: each control code moves the clock edge a little further along. So there is no frequency response worth fitting, and comparing voltages only measures noise. Conventional flows have nothing to offer this block.

spice2rnm recognises what the block does and changes its approach. It measures how far the edge moves at each control code and builds a delay-line model from those measurements. It then generates a *phase* testbench: one check per control code against the value measured in SPICE. The testbench has no DC or AC section, because neither tells you where an edge lands.

Codes measured	9 of 9
Total edge movement, code 0 to 8	−500.1 ps, monotonic
Mean step (1 LSB)	−62.51 ps
Worst step error (DNL)	−100.5 ps (−1.61 LSB)
UVM-MS phase section	9 checks, 0 failed
worst	3.26 ps of a 6.25 ps tolerance

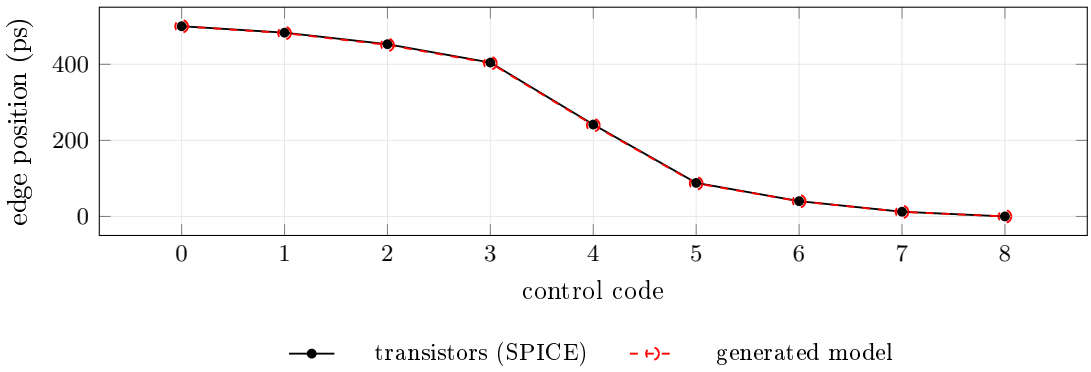
Two terms in that table need defining. **One LSB** (least significant bit) is one step of the control code. The nine codes span 500.1 ps in eight steps, so the average step is 62.51 ps. That average is the natural unit for everything else here. **DNL** (differential non-linearity) is how far each individual step is from that average.

The steps in this circuit are very uneven. Subtracting each measured edge position from the next gives eight steps of −17, −30, −48, −163, −153, −48, −28 and −12 ps. Each should be −62.5. The worst is the fourth step: it moves 163 ps for a single code change, 100.5 ps more than it should. That is the −1.61 LSB in the table.

This unevenness matters in a real system. A phase interpolator usually sits in a control loop that moves a sampling clock one code at a time to keep it centred in a data eye. That loop assumes each code change is worth about one LSB. Here one code change moves the edge 12 ps and another moves it 163 ps, so the loop's gain varies by more than 13× depending on where it starts.

The pass/fail tolerance per code is a tenth of an LSB, the finest step the model can resolve. That is tight enough to catch a real error, and loose enough never to fail on rounding. The uneven steps are the circuit's own imperfection, reported exactly as measured. The model's job

is to reproduce them, and it does. A model with evenly spaced steps would look tidier, but it would hide a real defect from anyone verifying against it.



Edge position at each of the nine control codes: the transistor-level circuit in SPICE (black) against the generated model (red). There is no waveform to compare here. What this block does is place a clock edge, so edge position is what is plotted.

6 Case study 4: Phase-locked loop

The first three case studies are single blocks. This one is a *loop*, and that changes what a model has to get right. In a loop, a small error does not stay small. It is fed back around and moves the whole system to a different operating point. So a model that looks reasonable on its own can still settle the loop somewhere the real circuit never goes.

The netlist also holds three different characterisation problems, and no single method covers all of them. The blocks are the netlist's own `.subckt` boundaries:

```
.subckt ro_vco aout cont dd
  ...                          (supplies, and the wires closing the ring)
  xinv1  dd ss sub well cont inm1  inp1  outm1 outp1 gaincell
  ...                                   (7 ring stages)
  xinv11 dd 0  sub well outm1 outm2 inv1
  xinv12 dd 0  sub well outm2 aout  inv2
.ends ro_vco

xcp  pupb pdn dra dd  cpump
xlf  dra vout         lpfilt
xvco aout vout dd     ro_vco
```

The top level has three `x` lines (subcircuit instances), so three models are joined into the runnable whole. `inv1` and `inv2` are instantiated one level down, inside `ro_vco`. They get their own models and their own pass/fail results, which is why the score below is five of five and not three of three. They are not wired into the composed top, because the `ro_vco` model was measured with them already inside it. `gaincell` gets no model at all. It has two inputs and two outputs, so no method that fits one input to one output can characterise it, and the composed model does not need one.

Each remaining block is classified from its topology and given a *route*, meaning the characterisation method chosen for that block:

Block	Why the default does not fit	Route taken
`ro_vco`	oscillates with its inputs at DC; its output is a frequency, not a gain	tuning curve, scored on rate
`cpump`	a complementary pair driven from independently controlled gates	one $I(V)$ table per control state
`lpfilt`	only R and C, so its state space is exact and needs no fitting	exact state space, read from topology

None of the three is a simple input-to-output response. A conventional flow that fitted one to each block would fit the wrong quantity three times.
Appendix A explains, with this circuit's own numbers, what a tuning curve, a control state and a state space are, and why an oscillator is compared by its frequency (its rate) rather than by its waveform.

6.1 Checked as a system, against the transistor-level circuit

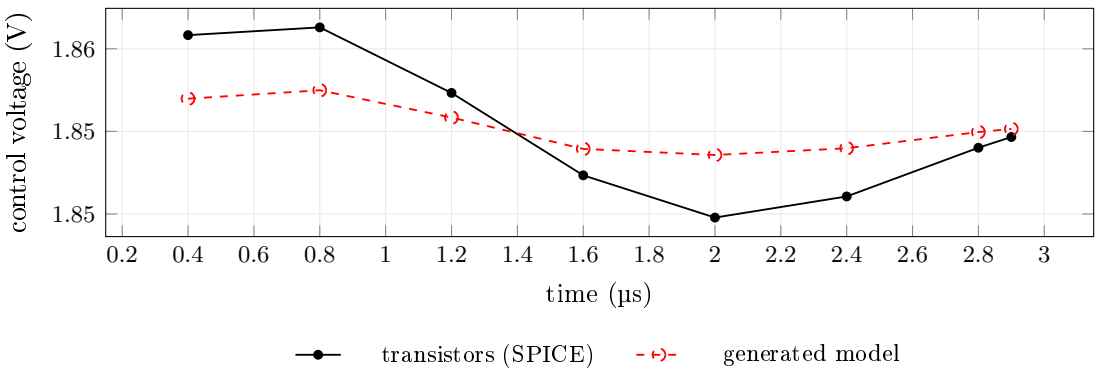
A score for each block tells you about that block alone. A loop raises a different question: wired together, do these models behave like the circuit? An SV–SPICE co-simulation answers it directly. Here that means the digital RTL runs in the SystemVerilog simulator while the analog half runs as the transistor-level circuit in SPICE (ngspice), joined by a bridge.

The design's own testbench is run twice. In the first run the analog half is the generated models, all in SystemVerilog. In the second it is the transistors in ngspice, in co-simulation with the digital RTL. Only the small piece of code joining the two halves changes between runs. The thresholds, expressions and pass/fail logic are byte-for-byte identical, so the two logs can be compared line by line.

Veylon Systems

Blocks modelled, three routes	3 composed, 2 more generated
Per-block equivalence	5 of 5 pass
Supply noise	20 mV rms, declared in the netlist
System check vs transistor-level circuit	4 boundary signals, 0 failed
`vout` (control voltage)	3.851 mV, 41% of its pass/fail limit
`aout` (VCO output)	0.4% of crossings, limit 2%
`dra` (internal probe)	19.1 mV, 0.8% of its range
Both loops lock at	400.00 MHz
control voltage range, transistors	1.8448–1.8563 V
control voltage range, models	1.8486–1.8525 V
Model half vs transistor half	4.81 s vs 76.2 s
same testbench, same span	16×

The control voltage is the number to read. The loop stores its state there, so a model that settles in the wrong place shows it here first. The two runs agree to 3.851 mV and reach the same frequency. They do so with 20 mV rms (root mean square) of random noise on the supply, as the netlist declares. The agreement is looser than it was with a quiet supply, and the supply noise is the reason: a loop concentrates a supply disturbance onto exactly this node.



The control voltage at the boundary between the digital RTL and the analog half: the composed models against the transistor-level circuit in SPICE, on the design's own testbench, on a node the loop holds near 1.85 V. Both settle to the same place and hold it. There are eight points because the digital side reads this node eight times. Because the supply carries random noise, how closely the two curves agree at any one instant is one run's result, a random sample. The pass/fail limit in the table is what the result is checked against.

The agreement figure changes from run to run. The pass/fail limit does not.

The supply noise is random. Each run generates its own random noise from the noise source declared in the netlist, so the model run and the transistor run never see the same noise. The largest difference between them, over any time window, therefore changes from run to run. Seven consecutive runs of this same netlist gave 3.61, 1.83, 4.15, 2.05, 1.86, 1.72 and 4.62 mV, and the run this section is drawn from gave 3.851 mV as an eighth. Two of the eight were run back to back with nothing changed between them. So the 3.851 mV quoted above is one random sample from a spread of possible results, not a property of the model.

What stays fixed is the limit the difference is compared against. That limit is 0.252 of `vout`'s own scale, and 0.202 of it is there to allow for exactly this run-to-run variation. The 3.851 mV comes to 0.104 against it, comfortably inside. That ratio is what the pass/fail result is based on, and what the record keeps.

The scale has a minimum of its own: 2% of the node's magnitude. Without that minimum the comparison would be unfair here. In the transistor run `vout` moved only 11.5 mV across the window, and measuring a difference against so small a range would report ordinary noise as error.

In general, an agreement figure quoted without its limit tells you less than it seems to. A jitter ratio means little without the correlation time it was measured at, for the same reason.

Each node also records which metric its limit applies to. The digital side reads some nodes through a threshold. For those, what matters is when the signal crosses the threshold, not its average value, so they are scored on their crossings. Printing the average beside a crossing limit would show a pass next to numbers that seem to contradict it (0.030 against a 0.020 bar) when the figure actually compared was 0.004.

6.2 The supply noise the netlist declares is the noise the model sees

The netlist for this case says

```
vdd dd 0 dc {vcc} trnoise(0.02 1e-10 0 0)
```

This is a supply with random noise on it, set by the `trnoise` (transient noise) settings. The first two numbers are the white-noise settings: an amplitude of 20 mV rms, and a time step of 100 ps between new random values. The next two, both zero here, would add flicker (1/f) noise.

A composed model that reads that line and generates `assign dd_v = 3.3;` has deleted a disturbance the design asked for. Every later check then agrees about a circuit nobody is shipping. The tool instead generates this, in `netlist_top.sv`:

```
int unsigned dd_rng;
initial dd_rng = DD_SEED;
real dd_nw;   // white, sample-and-hold at the deck's NT
initial begin
  dd_nw = 0.0;
  forever begin
    #(DD_NOISE_DT_S * 1.0e15);
    dd_nw = DD_NOISE_RMS * __gauss(dd_rng);
  end
end
assign dd_v = ((dd_v_ext > 0.0) ? dd_v_ext : DD_NOM) + dd_nw;
```

The noise amplitude and the time step are the ones the netlist declares. The supply can still be overridden from outside. The random generator starts from a fixed random seed (`DD_SEED`), so a rerun repeats exactly. In a transient simulation of this netlist, that noise source causes 10.72 ps rms of period jitter at the oscillator, ±5% over the two hundred cycles it was measured on. That jitter figure is itself one random sample. A constant supply produces no jitter at all.

The jitter figure is measured for each design, not quoted from elsewhere. Measuring it needs one step first. To characterise a block, the tool extracts it from the netlist, and extraction rebuilds every supply as a plain DC source, without the declared noise. So the tool restores the declared noise onto the block's own supply. Without that step the measurement would describe a quieter circuit than the one being modelled. The generated file records which source the number came from.

The declared source is therefore carried into the composed model: a new random value is taken at each of the source's own time steps and held until the next. That reproduces the two properties that decide how much jitter a noise source causes: its rms and its correlation time. The noise spectra of model and SPICE agree at low frequency. They differ near $1/NT$ (NT is the time step), because ngspice interpolates between its random values and the model holds them flat.

All seven fields of a `trnoise` declaration are read, not just the two white-noise settings. Where a netlist declares flicker (1/f) noise, the composed model carries it as a Voss–McCartney summation (a sum of random sources updated at octave-spaced rates) across the octaves the run spans. Where it declares random-telegraph noise (a signal that jumps between two levels), the model carries it as a two-state Markov process with exponentially distributed dwell times, using the capture and emission times given. The Voss construction produces $\alpha = 1$. The generated file records both that and the exponent the netlist asked for, rather than quietly substituting one for the other. This PLL's netlist declares white noise only, which is why every figure here shows white noise. Flicker noise dominates close-in phase noise, and a netlist that declares it gets it in the model.

Carrying the noise changes what a comparison means. The model run and the transistor run now each generate their own random noise. Two runs of the same circuit therefore differ by chance alone, and with a fixed tolerance the pass/fail result would depend on how long the run was. So each node's limit is widened to allow for that run-to-run variation: by three standard errors of the difference between two window means. The standard error is taken from how much the SPICE reference itself varies, not from the declared amplitude, because how much noise reaches a node is a property of the circuit. On `vout` that allowance is 0.202 of the 0.252 bar. The record carries both, so a reader can see how much of the bar was earned by agreement and how much was allowance for noise.

6.3 Where the block works is measured, not assumed

The two output-buffer inverters sit in the clock path. Their job is to place clock edges in time, and an average voltage error barely sees that. Both pass an average-error comparison comfortably, yet their error at the single worst instant is 99.9% of the full voltage swing. These

do not conflict. If the model's edge lands slightly late, then for that brief moment the model sits at one supply rail while the real circuit sits at the other. Averaged over time, that moment disappears. For a clock buffer, that moment is exactly the behaviour being verified.

The tool classifies the two inverters from their measured DC curve. With 62% and 61% of swept points within 5% of a rail, the curve is a switching characteristic, not a smooth response. So they are scored on duty cycle instead.

The clock frequency they are scored at is also measured. Where a block operates is a property of the closed loop, and exists only once the digital side is attached. The same holds for the operating voltage range, which the golden probe (the SPICE run that measures golden values, the expected values) already measures. So the probe measures each net's switching rate beside its voltage range. The two inverter inputs return 399.8 and 399.7 MHz, the 400 MHz this PLL locks to. Scored there, they pass at 0.276 and 0.307 pp (percentage points) of duty cycle, against a 0.5 pp bar.

The model form they are regenerated in has no delay line, so it cannot represent the measured rise/fall skew: 0.589 and 0.672 pp of duty at that clock. Both blocks measure below their own floor, so the skew partly cancels rather than adding. That makes the floor an upper bound on what the simpler model form costs, not spare margin. The tool states this before scoring, so a pass is not read as unqualified.

A language model cannot supply that clock frequency. Asked five times about the same inverter with the same measurements, the classifier (the language model that judges design intent) proposed clock rates of 2 MHz, 765 MHz, 300 MHz, 10 GHz and 15 GHz. Asked about the second stage, it proposed 10 GHz. The lowest is two hundred times below where the buffer runs. The highest is thirty-seven times above what the buffer's own model can follow. The measurement returns the same answer every run, and its answers for the two stages agree to 0.03%, which no classifier run did. The rule is: design intent comes from the language model, physical values from the circuit. The clock rate looked like intent but is physics.

On what it is actually asked for, the classifier has been consistent: buffer, clock stimulus and duty-cycle metric, on every run of both stages. That is a judgement it can check against a measurement. What varied is the one number it had no way to know.

The pass/fail result names the metric it came from. Asking for the classifier does not guarantee it is used. Every duty-cycle figure above comes from a run with the classifier reachable and consulted. The same demonstration was then run deliberately with it *unreachable*, to see what a reader would be shown in that case. (Which credential makes it reachable is a property of the build, not of this result: the demonstrations ask the toolchain whether it can call a model rather than testing for a named environment variable.) The default chirp stimulus is kept, and the same two blocks pass an rms comparison at 0.025 against a 0.15 bar. That is a green result from a measurement that cannot see the property in question (edge timing). The two transcripts are otherwise alike and the command line is identical. So the run records which figure each pass/fail result was based on, and the demonstration reports it from that record. Asking for a check on the command line does not prove that it ran.

6.4 The same loop, in SystemVerilog alone

Everything earlier in this case study depends on an SV–SPICE co-simulation set-up already existing: the digital RTL runs in the SystemVerilog simulator, the analog half runs as transistors in ngspice, and a bridge joins them. Two steps rely on it. The golden probe is a run of the design's own testbench in that co-simulation, against the transistor-level circuit in SPICE, which measures the range each block really operates over. The system check runs that same testbench twice. Both need a testbench spanning the two halves, a DPI bridge, and a shared `libngspice` for the bridge to load.

Most designs have none of those. What they have is an analog netlist, some RTL, a specification in English, and nothing wiring the three together. This section runs the same PLL that way, in SystemVerilog only (RTL plus generated models, no SPICE anywhere). Call it the SV-only run, to keep it apart from the SV–SPICE co-simulation run earlier. In the SV-only run, two things break.

Nothing measures the operating band. A block's operating band is the range of voltages it actually sees in use. That range is a property of the closed loop, and with the loop open there is no loop to measure. Worse, two of the ring oscillator's internal nets are driven by nothing at all, so they have no operating point either.

The band therefore comes from the two artifacts that do know it, and the run says which for every net. Most of it is in the *deck*, and is read rather than restated: a source holding a node at one voltage is that node's band, and a node with a pull-down to ground and a pull-up to the rail cannot leave them. Two numbers are in neither the netlist nor derivable from it: where a *locked* loop keeps its control voltage, and how far the supply is allowed to move. Both are *declared*, from the operating-conditions table a designer writes anyway. Seven messages covering twenty-one nets, five of them facts about the circuit and two of them design intent:

```
band for 'aout'       DERIVED   as 0..3.3 V, the range its own drivers
                                can reach -- not where it sits; the same
                                for 14 more net(s): xvco.outm1 ...
                                xvco.outm7, xvco.outp1 ... xvco.outp7
band for 'xcp.ss'     DERIVED   as 0 V, fixed: source vss ties it to 0 V
band for 'xvco.ss'    DERIVED   as 0 V, fixed: source vss ties it to 0 V
band for 'xvco.sub'   DERIVED   as 0 V, fixed: source ve ties it to 0 V
band for 'xvco.well'  DERIVED   as 3.3 V, fixed: source vpe ties it to 3.3 V
band for 'dd'         DECLARED  as 3.13..3.47 V from the specification's
                                'Supply (VDD)' row -- not measured
band for 'vout'       DECLARED  as 1.75..1.95 V from the specification's
                                'Control voltage' row -- not measured,
                                because no golden run was available to
                                measure it
```

Nothing on the command line states any of this, and nothing in the specification names a net. The document says "control voltage"; the netlist says `cont`; the instance card says `cont` is the top level's `vout`. What binds the three is the measurement that found which port tunes the oscillator, so the binding happens after classification and is reported with the block and port it came from. Earlier these five numbers were five `--band` flags on the command line, which is the one place a number has no owner: it cannot be traced to the document that justifies it, it is not reviewed with the design, and a copy of it in a script drifts from its source silently.

One further distinction the run keeps. A band read off a pair of rails is what a net *can reach*, not where it sits, so it is the weakest of the three and never narrower than the truth. The charge pump's output can reach either rail too, and in a locked loop it sits at the control voltage instead; that topology is recognised and left without a band rather than given a true and useless one.

A band is an input, not a claim. The operating-conditions table is parsed by text, with no model involved, while the rest of the document is read by one. The asymmetry is deliberate. Every other fact taken from a specification is checked afterwards against what the circuit was measured to do, and the measurement wins; a band cannot be checked that way, because it decides the range the circuit is measured over. A number with that much authority must not depend on whether a model could be reached, so it is read by the part of the toolchain that always runs.

The tool also reads the specification itself. The demonstration gives it the specification

document along with the netlist and the RTL. This is what it took from that document, in the run these figures come from:

```
read as: Document states "PLL -- 400 MHz ring-oscillator
  phase-locked loop" with "seven-stage differential ring" whose
  "behaviour is a frequency, not a gain" -> oscillator/vco
  environment; "Output ... 400 MHz" is the highest stated
  frequency; "Reference ... 10 MHz" is the lowest stated rate;
  "20 mV rms of wideband noise is specified on it"; "hold 400 MHz
  to within 2%" and "No small-signal specification ... Do not
  verify any of them against a frequency response" implies
  timing-based verdict; control-voltage swing range quoted as
  "characterizable from 0.5 V to 2.8 V".
```

These are facts about what the part is FOR, which is a different kind of statement from the operating band above: they choose the stimulus and the verdict, and they are claims the measurement can contradict. Section 10 describes the rule that governs them: every fact extracted from the document is checked against what the circuit was measured to do, and the measurement wins. Here nothing disagreed.

More useful is what it refused to let pass. The document names 20 mV rms (root mean square) of supply noise as a condition the part must work under. The reader placed that correctly: an operating condition belongs in the stimulus, not in the model. It then reported that the generated environment does not drive that noise:

```
spec: the document states 20 mV rms of supply noise as a condition
  to work under. That is a property of this block's environment, so
  it belongs in the stimulus rather than in the model
NOTE: the supply-noise condition is not driven by the generated
  environment yet; the composition carries only the noise the
  netlist itself declares
```

So a requirement the tool cannot yet meet is flagged in the output, not silently dropped. That is the behaviour worth having from a language model in a verification flow: not a guess at a fix, but a gap it can name.

It did not supply the five bands. A person still reads those off the specification's table and types them on the command line. The run prints the phrase `a spec DECLARES` for them. That phrase is the band mechanism labelling a figure a human supplied. It does not mean the reader extracted that figure.

This passage costs the paper some reproducibility. Every other figure in the paper regenerates identically from its run. The quoted text above does not, because a language model produces different words each time. What stays stable is everything around it: the bands, the measurements and the verdicts. All of these were byte-for-byte identical across this run, two earlier runs where the model read the document differently, and the run before those, where the reader did not run at all because no model was reachable. What a model changed was what was *read*, never what was measured.

Neither a derived band nor a declared one is a measured band, and the run never presents either as one. Every message that uses a band names its source in those words: `a spec DECLARES as 1.7500..1.9500 V, not measured` for the control window, and `the deck itself gives as 0.0000..3.3000 V` — a complementary pair drives it between the rails — `derived and not measured` for a net inside the ring. The blocks the open loop cannot reach say so too, instead of quietly characterising something else.

Per-block models are still not a system. Checking each block alone does not show that the loop works. Without an SV–SPICE co-simulation that question still needs an answer, so the SV-only run closes the loop in *pure SystemVerilog*. It uses the composed models and the

design's own `pfd.sv` and `divn.sv`, instantiated unchanged, in one simulator. There is no DPI import, no bridge, and no SPICE process anywhere in it.

The user states no *topology*. An earlier version of this demonstration shipped a JSON file naming which RTL port landed on which net, and that file is gone: the wiring is followed out from the one clock the analog half produces, by port *direction*, and not one port name is read on the way.

That last part is the point, because on this circuit a port name is actively misleading. In ngspice's own PLL example the detector is instantiated `Xfpdet d_divout d_ref` against a subcircuit declared `.subckt f-p-det d_R d_V`, so `d_R` carries the *divided* clock, the opposite of what it reads like. A loop wired from names is confidently backwards there. What the run worked out here, and what from:

```
pfd  . ref_clk -> clk_out      divn . clk_in  -> aout_clk
pfd  . div_clk -> ref_clk      divn . clk_out -> clk_out
pfd  . dn      -> dn           analog aout    -> aout_clk
pfd  . up      -> up

the oscillator's control net = vout
    the port block_classify measured as its control source
the net that becomes a clock = aout
    block 'ro_vco' was measured to oscillate on its port 'aout'
the divide ratio = 40
    parameter N of module divn defaults to 40 in its own header
the reference rate = 1.00014e+07 Hz
    the deck holds 'vout' at 1.8495 V, where the measured tuning
    curve puts block 'ro_vco' at 400.056 MHz, and the ratio is 40
```

One fact about this loop is in neither the netlist nor the measurements. Which of the detector's two outputs speeds the oscillator up is a property of the RTL's *body*, and both wirings elaborate and run. This used to be settled by building both loops and keeping whichever locked, which works, costs a second loop run every time, and leaves the question answered by an experiment rather than by the document that ought to state it. The specification states it instead, in one line that names a port of the customer's RTL and the control voltage, and no gate of the charge pump:

> **The detector's `up` output raises the control voltage.**

That is read by text, with no model and no credentials, for the same reason the operating-conditions table is: it decides how the loop is *wired*, so it must not move with whether a model could be called.

And "raises the control voltage" is not "speeds it up". This ring's tuning curve falls, so the output that raises the control voltage is the *slower* one, and the run does that conversion with the sign it measured rather than assuming the two agree. Reading one as the other is exactly the error the statement exists to remove, and on this circuit they are opposite.

The stated sense does not outrank the circuit. The other pairing is still written out beside the one that was built, and if the stated one fails to lock it is built and run too: a loop that locks is a measurement and a document is not, so the measurement wins and the run reports the document as wrong about its own detector. On this deck the stated pairing locked first time, so one loop was built and the alternative stayed a record.

The user does *not* state the polarity of either charge-pump gate (whether it is active high or active low). The tool derives it from two things characterisation has already measured: the sign of the oscillator's tuning curve, and the direction of the current each control state drives. The generated file records the numbers it came from:

```
pdn <- dn, active HIGH (derived)
  ro_vco measured at 15 control points, 0.495 to 2.805 V, over which
```

```
    its frequency FALLS 804.47 -> 16.74 MHz: Kvco is NEGATIVE
  lpfilt: current into dra reaches vout through a state space whose
    input vector is non-negative throughout, so positive current
    RAISES it
  with cpump quiescent (pupb=1), pdn at 1 drives -0.00447 A into dra
    and at 0 drives -1.961e-09 A, both at 1.849 V -- active HIGH
pupb <- up, active LOW (derived)
  ... the same two measurements, and with pdn=0, pupb at 1 drives
    -1.961e-09 A into dra -- so its asserted level is the LOW one
```

On the same pump, `pupb` comes out active *low* and `pdn` active *high*. A port name does not reliably carry that distinction, and charge pumps wired from their names have shipped upside down before.

The table summarises the SV-only run. "Route" here means the characterisation method the tool chose for a block.

SV–SPICE co-simulation, at any stage	none
Models generated, using four routes	5
Operating bands	5 derived, 2 declared, 0 measured
Per-block equivalence	5 of 5 pass
Loop closed in	SystemVerilog only
Loop sense, stated by the specification	1 line, no net named
Gate polarities derived from measurement	2 of 2
Gate pairings built and run	1 of 2, the stated one
Divider output vs the reference it follows	10.001348 vs 10.001401 MHz
against a 2% pass/fail limit	−0.0005%, locked
Control voltage across the window	1.8488–1.8506 V
drift end to end	−0.26 mV
Environments generated	3 per block, 1 over the loop
blocks refused, with a reason	2
run, and reporting results	4 of 4
Characterisation, loop included	11 min
The four environments, run afterwards	11 min

A UVM-MS environment over the whole loop. The loop above is a plain testbench: it drives a reference clock and checks a frequency. The SV-only run also generates a full UVM-MS environment around it. It has a sequence, sequencer, driver, monitor, agent and scoreboard. It also has an MS proxy split in two: an abstract half that the components talk to, and a concrete half in the top level. As a result, neither the driver nor the monitor holds a hierarchical path into the design.

It is the only *system-level* environment this tool generates with all of those components. The reason is the stimulus:

environment	seq	drv	sqr	mon	agt	prx	scoreboard
block, transfer function	•	•	•	•	•	•	`uvm_scoreboard`
block, oscillator	•	•	•	•	•	•	`uvm_scoreboard`
system, SV–SPICE co-simulation	—	—	—	•	•	•	`uvm_scoreboard`
system, SV only	•	•	•	•	•	•	`uvm_scoreboard`

The system environment for the SV–SPICE co-simulation run has the observing half and none of the driving half. The testbench it wraps brings its own stimulus, so there is nothing for a

sequence to ask for. The SV-only run has no such testbench. Its stimulus is the reference clock, and this environment generates it, so a sequence, a sequencer and a driver have real work to do.

The oscillator's block environment uses the same pattern, and the pattern is what makes both extensible. The timing measurement lives in the proxy's concrete half in `top`, the only scope that can see the design's signals. The driver, monitor, agent, scoreboard and test hold no references to either signal, against eight in `top`. A second observer can subscribe to the monitor's analysis port without restructuring anything. And *one* implementation of "time a clock" serves both this environment and the loop one.

The reference rate itself is not a round number any more. It is the measured tuning curve read at the control voltage the deck states, divided by the ratio the divider's own header declares: 1.8495 V puts the ring at 400.056 MHz, and 40 of those make 10.001402 MHz. The loop environment's sequence and driver use that timing to step the reference frequency up by 0.5% and back. That is why the results below have three measurement windows, not one settled snapshot. A model frozen at its initial condition would produce a single settled snapshot too, so one snapshot proves little.

Its expected values come from two places, and every line of its report says which. Two come from the specification: reach the reference frequency within tolerance, and keep the control voltage inside the stated window. Two come from this run's own measurements, and the second of those is the valuable one. The oscillator's tuning curve was measured point by point in ngspice before any of this existed. Inverting that curve at the frequency the loop is asked for gives the control voltage the oscillator needs to produce that frequency.

Window	divider output	control voltage
nominal, 10.001402 MHz	10.001342	1.8488–1.8505 V
stepped, 10.051409 MHz	10.050600	1.8435–1.8457 V
returned, 10.001402 MHz	10.000518	1.8490–1.8506 V
rate, against a 2% bar	3 checks, 0 failed, worst 0.0088%	[SPEC]
control window, 1.75–1.95 V	3 checks, 0 failed	[SPEC]
against the measured curve	3 checks, 0 failed, worst 0.89 mV	[MEASURED]
boundary finite	3 checks, 0 failed	[MEASURED]

The stepped window is the one to read. The reference went *up* and the control voltage went *down*, which is what a negative K_{vco} requires. It also went down by the amount the measured curve predicts. From its 15 points, that curve puts 400 MHz at 1.8496 V, with ±2% spanning 1.8291–1.8705 V. The loop settles 0.89 mV from the prediction, inside a window 41.5 mV wide.

That check is worth reporting because it can fail. A loop can reach the right frequency at the wrong control voltage, for example through a charge pump with the wrong sign, a wrongly fitted slope, or a divider ratio that the reference happens to absorb. Counting clock edges cannot see that. Shifting the measured table by 100 mV makes exactly those three checks fail, by about that same 100 mV, while the two specification checks still pass. An environment built only from the specification would have passed the same run.

Three per-block environments are generated beside it: one for each of the two clock buffers, and one for the oscillator. Two blocks are refused, with the reason written out. The charge pump's result is a current for each control state, and the filter's is a voltage produced by an injected current. Every generated environment, however, scores a voltage against a stimulus, so neither fits.

All four are then run, which is a stronger claim than all four being generated. A file count shows that a generator produced something. It says nothing about whether the output elaborates, compiles against the Accellera UVM library, or scores anything. Each environment is executed and its scoreboard result reported:

Environment	what its scoreboard concluded	time
`inv1`	54 DC checks, 0 failed, worst 0.001 mV of a 6.6 mV bar	6 s
`inv2`	54 DC checks, 0 failed	111 s
`ro_vco`	15 breakpoint + 5 held-out, 0 failed, worst 0.024%	219 s
the loop	12 checks over 3 windows, 0 failed	336 s

The two inverter results are the weakest of these, and the last paragraph of this subsection explains why. All four environments are published. All four were run from the published copy, not from the run directory, so what is published is exactly what ran.

What this result does not carry. There are two limits, and neither is small.

First, there are no transistors in this loop, so nothing in the SV-only run is evidence about the circuit itself. A loop that settles at a plausible but wrong point still settles, and only the SV–SPICE co-simulation run earlier can catch that. The verdict covers this modelled loop, at this reference frequency, starting from the initial condition the netlist states.

Second, the SV-only run's 5 of 5 is not the same 5 of 5 as the SV–SPICE co-simulation run's. A clock buffer's verdict is only meaningful with the duty-cycle metric, and that metric needs the clock's frequency. The golden probe is what measures that frequency, and the SV-only run has no golden probe. So `inv1` and `inv2` fall back to comparing average voltage: 0.021 and 0.026 against a 0.15 bar. Both pass, but this check is blind to where the clock edges fall, for exactly the reason Section 6.3 gives. The two runs agree on the count and differ on what the count is worth.

7 Results at a glance

Case study	Environment	Checks	Failed
Filter	DC + AC	49 + 15	0
Duty corrector	DC + duty	46 + 14	0
Phase interpolator	phase per code	9	0
Phase-locked loop	SV–SPICE co-simulation vs SPICE	5 + 4	0
the same loop, SV only	UVM-MS loop env, derived/declared bands	5 + 12	0

The first three case studies run end to end in under 16 minutes on a laptop. That covers characterisation, model generation, both simulators and every generated environment. The PLL takes a further 11 minutes, mostly to measure the ring oscillator's tuning curve (its frequency against control voltage).

The SV-only variant takes time for the same reason. On the run these figures come from the pipeline took 11 minutes, closing the loop included, and running the four generated environments afterwards took a further 11. The total varies between runs, because the oscillator's refinement passes vary and because the machine is shared. Its twelve extra checks belong to the loop environment and cover three windows. The rate and control-window limits come from the specification. Agreement with the measured tuning curve, and finiteness (no NaN or infinite values), come from this run's own measurements. Its 5 per-block passes carry less weight than the 5 in the row above, for the reason given in Section 6.4.

Each case is one command: `demo_4_pll.sh` for the PLL checked by SV–SPICE co-simulation (the digital RTL in the SystemVerilog simulator, the analog half as transistors in ngspice, joined by a bridge), and `demo_4b_pll_nocosim.sh` for the SV-only row beneath it. The co-simulation case is run the same way as the others, but it needs more installed. As the analog half of a co-simulation, it needs the other half: the RTL, the DPI bridge, and a *shared* libngspice. That is a different ngspice build from the standalone binary the first three cases use.

The script checks for each of those pieces before doing any work. If one is missing, it skips and says which piece is missing and where to get it, rather than failing twelve minutes into a run.

spice2rnm's outputs are plain IEEE 1800 SystemVerilog, tied to no simulator vendor. So that anyone can reproduce the results here, all of them were produced with open-source simulators. **ngspice** ran the characterisation and the reference transient simulations. **xezim**, an open-source IEEE 1800 simulator that runs the Accellera UVM library[1], ran both the SV–SPICE equivalence co-simulation and the generated UVM-MS environments. No commercial licence was used anywhere in this document.

The generated files are published for inspection, exactly as the tool wrote them: the models, the complete environments, and the evidence files. They are at

`https://github.com/vrajeshprakhya/spice2rnm-demos`

under `showcase/`, stamped with the revisions of the tool, the demos and the simulator that produced them. The repository holds:

```
spice2rnm-demos/
  README.md           what this repository is, and is not
  .gitignore          run scratch; the showcase is curated, not
                      whatever a run left behind
  demo_1_lpf.sh  demo_2_dcc.sh  demo_3_pi.sh
  demo_4_pll.sh  demo_4b_pll_nocosim.sh  run_all.sh
                      the five case studies, each one command;
                      run_all.sh chains the first four
  netlists/           the four SPICE inputs, exactly as the demos
                      run them: rc_lpf2, dcc, pi_therm, and the PLL
                      netlist with the supply-noise card it runs under
  specs/              the three specifications in English: the
                      filter's, the duty-cycle corrector's (demo 2
                      reads it) and the PLL's (demo 4b reads it)
  harness/            9 files: 8 measurement scripts the demos
                      call, plus the JSON stating how the PLL's
                      loop is wired
  showcase/           the generated artifacts, byte-for-byte
    lpf2/             the baseline case, 13 files + equivalence/:
      rc_lpf2_rnm.sv          the model, wreal ports
      rc_lpf2_ms_types_pkg.sv nettype real rc_lpf2_wire,
                              resolved by summing its drivers
      rc_lpf2_dms.sv          interconnect ports, straight through
      rc_lpf2_bridge*.sv      MS bridge and its driving core
      rc_lpf2_proxy_pkg.sv    agent <-> bridge API
      rc_lpf2_ms_pkg.sv       item, sequence, driver, monitor,
                              agent, and the SPICE goldens
      rc_lpf2_ms_scoreboard.svh  the checks and tolerances
      rc_lpf2_ms_tb/test.svh  environment and test
      top_rc_lpf2_ms.sv       nets, DUT, bridge, test
      run_rc_lpf2_ms.sh       compiles and runs it
      result.json             fit, verdict, warnings
      equivalence/            what the check actually ran and found
    dcc2/             same shape, 14 files + equivalence/: no AC
                      section, adds duty goldens in dcc_ms_pkg.sv
                      and the run log pipeline.txt
    pi/               same shape, 13 files + equivalence/: an MS
                      bridge, proxy and agent like the others, with
                      phase-per-code goldens and no DC or AC section
    pll/              53 files + 9 under equivalence/. Case study 4,
                      checked against the transistors:
```

[1] `https://github.com/vrajeshprakhya/xezim`

```
      11 at the top  five block models, three interfaces, the
                     composed analog top, the node package,
                     result.json
      env/inv1/  12  and env/inv2/ 12 -- a full UVM-MS environment
      env/inv2/  12  each: item, sequence, sequencer, driver,
                     monitor, agent, scoreboard, env, test, MS
                     proxy and bridge
      env/ro_vco/ 10 the oscillator's, same components, goldens a
                     measured tuning curve rather than a DC sweep
      env/cosim/  8  the SYSTEM environment: the design's own RTL
                     and every block's model elaborate together,
                     goldens the transistor run's own statistics,
                     carrying a period-jitter monitor at 1 fs
    pll_nocosim/     58 files + 9 under equivalence/. The same
                     circuit run in SystemVerilog only:
      11 at the top  the same models, interfaces and composed top
      env/inv1/  12  env/inv2/ 12, env/ro_vco/ 10 -- as above
      env/rtl_loop/ 7 the LOOP environment. The only SYSTEM
                     environment generated with the stimulus half,
                     because it owns its stimulus: proxy package,
                     package, scoreboard, env, test, top, run script
      loop/       2  the plain harness it replaces: one SystemVerilog
                     file and the script that runs it
      rtl/        4  the customer's pfd.sv and divn.sv, the English
                     specification, and the loop's topology in JSON
    house_lib/       4 files, NOT generated: ng_ams_pkg.sv
                     declaring nettype real ng_anet, and two
                     hand-written models using it
    lpf2_house/      13 files: lpf2 again, joined to house_lib --
                     no types package, and rc_lpf2_rnm_ng_anet.sv
                     in place of the wreal boundary
    STAMP            per case: when it was curated, from which run,
                     at which tool and simulator revisions; and the
                     last full run's summary
    refresh.sh       regenerates everything; the only way this
                     directory is ever updated
  whitepaper/        this document, source and PDF
```

7.1 What the equivalence folders hold

Every case study publishes an `equivalence/` directory. It holds what the check ran, what it found, and a list (`MANIFEST.txt`) of the large files kept out of the repository.

Case	Scored on	Published verdict
Filter	rms voltage	0.0290 against 0.15
Duty corrector	rms voltage	0.0769 against 0.15
Phase interpolator	phase per code	worst 1.832 ps at code 2, tolerance 6.251 ps
Phase-locked loop	five routes, then the whole system	five block verdicts, and the system over 116,018 samples

In the table, "rms" is root mean square: the rms voltage error between model and SPICE, against a pass/fail limit of 0.15. A "route" is the characterisation method chosen for a block; the PLL has five blocks, so five routes.

Start with two files. `equivalence/verdict.txt` summarises the check and its conclusion, one line per route: the metric, the worst point, the pass/fail limit (bar), and the verdict. The case's `result.json`, beside it, is the full record that line was taken from. It records what was driven, what was compared, over how many points, and every warning raised along the way.

Each folder carries three things. First, `verdict.txt`, copied by the tool from that run's own `result.json` rather than retyped, so a number here cannot disagree with the run that produced it. Second, the simulator and SPICE logs, which show what actually ran. Third, a comparison table with one row per measured point. For example, the phase interpolator's `phase_compare.csv` has nine rows, one per control code: the measured delay, the model's delay and the error. The PLL's `rc_compare.csv` is the loop filter's response to an injected current, sample by sample.

Why some files are listed in `MANIFEST.txt` instead of published. Some output files are large, easy to regenerate, and add little for a reader, so they are not committed. Git keeps every committed file in the repository's history forever, so anyone who clones it downloads them, even after they are deleted. Instead, `MANIFEST.txt` names each such file with its exact size and the command that recreates it. A reader can see what exists and how big it is, without every future clone carrying it.

The large files come from the waveform routes. These write one row per simulator timestep, not one per measured point. The filter's aligned comparison is 18.6 MB and its generated testbench 18 MB; the duty corrector's comparison is 2.8 MB. The PLL is the bulk: its SV–SPICE co-simulation waveform file alone is 202.8 MB, two thirds of everything the five cases produce.

Together these come to close to 296 MB, and `refresh.sh` regenerates all of them in half an hour. Committing them would make the repository permanently heavy for no lasting benefit. What is published comes to 1.4 MB. Of that, the SV-only PLL is 546 kB: four environments, the loop harness, the customer's RTL and both specifications.

The rule is based on file size, not file name. A comparison table small enough to read is evidence; one large enough to be a waveform is a trace. Routes differ in which they produce: a network compared point by point writes a table, while a waveform compared sample by sample writes a trace. So the rule has to look at the file itself, not at what it is called.

The repository also deliberately leaves out two things. One is the spice2rnm tool itself: the product is licensed separately, and this repository is its evidence. The other is the build products of the runs: characterisation scratch files, compiled simulator objects and DPI shims. `refresh.sh` filters those out, so the showcase holds what a user would keep, not everything a run produces.

8 What the equivalence check compares

Every model in this document was checked the same way. The generated SystemVerilog is loaded into a digital simulator and given an input signal. The transistor-level circuit in SPICE is given the very same input. The two outputs are lined up in time and compared point by point. Both traces are saved, so anyone can open the run's own folder and work out the number again.

Two properties of this check matter more than the numbers it produces. The first is that **it tests the file you are actually handed**. Some tools report a weaker number: how well the fitted equations matched the measurements they were fitted to. That number never opens the generated file, so it would look identical even if the file had been written out wrongly. This check loads the generated file into a simulator and runs it, so a code-generation bug makes it fail.

The second is that **each block is scored on its own job**. A single error figure used for every block would miss what matters on most of them. An average voltage error cannot see where a clock edge lands, and an oscillator's frequency has no voltage error to measure at all.

If the block	then compare	and score it as
passes a signal through	the output waveform, on a sweep and on a second, unrelated input	the error, against how far the output itself moves
drives a clock	where the edges land, at the rate measured in use	duty error, in percentage points
moves an edge per setting	the crossing time at every setting	picoseconds, plus step size and evenness
oscillates	the output frequency, as the control voltage varies	a percentage of that frequency
switches a current	the current it drives, in each switch position	a percentage, for each position
is passive R/C	the output's response to a current pushed into it	the error, against how far the output moves

The fourth row cannot be scored like the first. Two oscillators at the same frequency slowly drift apart in phase, so subtracting their waveforms would score a perfect model as entirely wrong. Appendix A explains this in full.

8.1 What keeps a figure from flattering itself

Some measurements are held back. When a block is characterised by measuring it at many points, a share of those points is kept out of model-building and used only for scoring. The headline figure is therefore not the fit marking its own work. The charge pump in case study 4 reports 0.549% error on held-back points; the ring oscillator reports its held-back frequency error directly.

An input the model has never seen. The filter in case study 1 is built from a frequency sweep. It is then tested with a pseudo-random bit sequence it was never shown (0.0097). A model that had merely memorised the data it was built from would be exposed there.

The report states the range the answer covers. A comparison proves nothing about conditions it never tested, so the report states the range it did test. The ring oscillator is the clearest case, and the honest answer there is two numbers. Inside the control-voltage band the loop actually operates in, its held-back frequency error is 0.024%. At the far edge of the wider characterised range, where the tuning curve bends hardest and where the loop never goes, the error reaches 4.987%. Both numbers are reported, and both come from the SV-only run of Section 6.4; the co-simulated run of the same block, whose band the golden probe measures instead of the specification declaring it, reports 0.001% and 4.231%. The pass/fail verdict is taken in the band that matters, and the less flattering number is printed, not quietly dropped.

Known limits of the model shape are costed in advance. Sometimes the chosen model shape simply cannot represent something the circuit does. When that happens, the cost is stated up front, in the same units as the verdict. An example is the inverter timing asymmetry in case study 4, quoted as duty error at the clock rate in use. A pass is then read against that floor, not taken as a clean win.

8.2 Blocks, and then the system

The table above scores blocks one at a time. Joining blocks raises a further question: errors that are small in each block need not stay small once everything is wired together. Three levels of check answer it.

First level: the netlist alone. The assembled model is driven at the external pins of the design. Its output is compared against the full transistor-level circuit in SPICE, driven the same way.

Second level: an existing SV–SPICE co-simulation. This applies when you hand the tool an SV–SPICE co-simulation set-up: the digital RTL runs in the SystemVerilog simulator, the analog half runs as transistors in ngspice, and a bridge joins the two. Case study 4 reports this level first. Here the design's own testbench is run twice: once with the generated models as the analog half, once with the transistor-level circuit. Nothing else changes. The digital logic, the limits it checks against and the pass/fail rules are byte-for-byte identical in both runs; only the small piece of code joining the two halves is swapped. That makes the two runs directly comparable, signal by signal: 4 signals crossing between the halves, 0 failed, the control voltage agreeing to 3.851 mV on a rail carrying 20 mV rms (root mean square) of declared noise, and both versions locking to the same frequency.

Third level: SystemVerilog only. Most designs have no SV–SPICE co-simulation set-up: just an analog netlist, some RTL, and nothing joining them. Here the models are composed and the customer's own RTL is wired around them in SystemVerilog only, with no bridge and no SPICE in the loop. The question is whether the assembled design does its job. A full UVM-MS environment drives it and checks it against two things: the specification, *and* what this run measured. For example, the oscillator's own tuning curve says which control voltage gives the target frequency. A loop that reaches the right frequency at the wrong control voltage is therefore caught, not counted as a pass. Section 6.4 runs the same PLL this way. It locks to -0.0005% of its reference, with the control voltage 0.89 mV from where the measured tuning curve puts it. This is deliberately a weaker claim than the second level makes, because there is no transistor-level circuit in the run to disagree with.

A check that does not apply to a given run is reported as not applicable, with the reason. It is never quietly counted as a pass.

8.3 One thing a score at an operating point cannot see

Every check described so far is *static*: it compares a value at a fixed operating point. That suits most of what a model claims. It is blind, though, to one error that matters: how the model responds to a control or supply voltage that is moving.

An oscillator model has to decide when to place its next edge. One way is to read the control voltage at the instant of the previous edge and schedule a whole half cycle from that single value (the *sampling* form). The other way is to integrate the control voltage across the cycle, which is what the real circuit does (the *integrating* form). Both forms use the *same* tuning curve. So a check that measures frequency at fixed control voltages scores them identically, and in practice it passes both. Feed them the same moving control voltage and they diverge. On the reference ring oscillator, with a control disturbance of 100 ps correlation time, the sampling form produced 3.37× the period jitter of the transistor-level circuit in SPICE.

The size of that gap is not a fixed property of the two forms; it depends on how fast the disturbance moves. The predicted ratio is $\sqrt{T_{\mathrm{half}}/\tau}$, where τ is the disturbance's correlation time (roughly, how long it stays at one value) and T_{half} is the half period. For 1.25 ns against 100 ps this predicts 3.54, which matches the measured value above to within 5%. A faster disturbance pushes the two forms further apart, so a ratio quoted without its τ means less than it appears to.

So the tool tests this dynamic behaviour directly. A known disturbance is injected in SPICE at several amplitudes. Then the exact disturbance waveform *that ngspice actually solved* is replayed into the model, sample for sample. Replaying matters. If the model were given its own, separate random noise of the same size, the check would compare two unrelated random sequences, and their agreement would depend more on run length than on the model. Replaying the same waveform tests how the disturbance is transferred into jitter.

Both of the oscillator's inputs are tested, not just the obvious one. On the reference ring, the supply voltage moves the frequency by 450.6 MHz/V: comparable to the control input, and opposite in sign. A model with no supply input could not show what a supply droop does at all. The overall verdict is the worse of the two paths.

In the table below, each row gives the rms (root mean square) period jitter of the circuit and of the model, and the model-to-circuit ratio. A ratio of 1 means perfect agreement.

Path	Injected	Circuit	Model	Ratio
either	none (floor)	0.051 ps	0.000 ps	no limit
control	5 mV rms	2.396 ps	2.352 ps	0.981
	20 mV rms	8.743 ps	8.690 ps	0.994
	50 mV rms	26.719 ps	25.618 ps	0.959
supply	5 mV rms	2.808 ps	2.640 ps	0.940
	20 mV rms	11.617 ps	12.139 ps	1.045
	50 mV rms	28.422 ps	27.656 ps	0.973

Each ratio is one random sample, like the boundary figures earlier, not a fixed property of the model. Every injection uses a new random noise waveform, and an rms measured over two hundred cycles has a sampling uncertainty of about 5% either way. That is why the same six points on the previous run read 0.987, 0.946, 0.925 and 0.896, 0.953, 0.970. This spread is why the pass/fail limit is 0.25 rather than something that looks stricter. A tolerance tighter than the measurement's own uncertainty would fail correct models, at a rate set by how long the run was.

The floor row is measured but shown for information, with no pass/fail limit. With nothing injected, the reference still reports about 54 fs of jitter. That figure is the precision with which the edge-finder locates a threshold crossing, not anything the circuit did. Asking the model to reproduce it would mean asking it to reproduce a measurement artefact. Instead, it is the floor that every other amplitude must clear before its result counts.

That agreement was not obtained by fitting anything; simple physics predicts it. Take the supply path, where the sensitivity is measured separately. If the disturbance held still for a whole cycle, 450.6 MHz/V on a 2.509 ns period would predict 14.18 ps of period jitter for 5 mV rms. The circuit gives 2.797 ps, a ratio of 0.197. The reason is that the oscillator averages a fast supply disturbance over its period. Averaging about T/τ independent pieces of noise shrinks the effect by $\sqrt{\tau/T} = \sqrt{100.3\,\text{ps}/2509\,\text{ps}} = 0.200$, which is what the circuit shows.

This holds across the tenfold range of amplitudes, not just at one point: the ratio is 0.197, 0.223 and 0.203 at the three amplitudes checked. An oscillator that did not average its input would sit at 1, five times higher. All three points are within 12% of a prediction with no fitted parameters, and each carries the same few per cent of sampling uncertainty as the ratios above. The integrating model reproduces this because it integrates; the sampling model does not.

The tool decides for itself whether this check is worth running. For most designs the honest answer is no, and the check costs a quarter of an hour. Two things can say yes.

The first is the design's own closed-loop behaviour, measured. The tool measures how far the control voltage moves within one oscillator period, converts that into period movement using the tuning curve's slope, and compares it with what the reference measurement can resolve. On this PLL the answer is no, backed by numbers. On a run where this was the deciding input, the control moved 12.18 µV rms per 2.503 ns period. That is worth 0.0288 ps against a 0.0521 ps floor, or 0.55× what the reference can resolve. A loop filter holds its control voltage still; that is its purpose.

The second is the netlist, and it is what triggers the sweep above. This netlist declares 20 mV rms of noise on the ring's supply, and a design that writes noise into its own netlist is asking for noise to be simulated. The rule matches on the net name, not the source name, because block extraction rebuilds every supply as a plain DC source. The extracted fragment carries

`VBIAS2 dd 0 DC 3.3`, where the top level wrote `vdd dd 0 dc {vcc} trnoise(...)`. The net name survives extraction; the source name does not.

There is no flag to force the check, because a request to run it is not evidence that it is needed. Where the tool has no closed-loop evidence either way, it runs the check rather than skipping it. Having no evidence that a check is needed is not the same as deciding it is not.

What the sweep measures, and what it does not. The sweep drives the ring oscillator in isolation, on its own characterisation fragment. That is the right scope for its question: does this model transfer a disturbance into jitter correctly? It is a different question from what the PLL does with a disturbance. Inside the loop, the filter holds the control voltage still, which is exactly why the closed-loop test above says no. These numbers come from driving the block directly, with nothing holding it. So the supply rows are not a statement about this PLL's supply rejection. They are a statement about the model.

9 Why a green testbench is worth believing

Generated verification has an obvious credibility problem. The same tool writes the model *and* the test. What stops it from writing a test that its own model is bound to pass? Three rules answer that.

The right answers come from SPICE, never from the model. Every expected value in every generated test environment is a golden value: a SPICE measurement of your circuit. Checking a model against the equations it implements would pass however wrong the model was. So spice2rnm never does it.

Every environment is proven able to fail. The tool deliberately breaks the generated model and confirms that the test goes red (fails). It also *measures* the smallest break the test still catches, rather than simply claiming one. The example below is the duty-cycle corrector's published environment, the one checked against its specification's 1.0 pp requirement (pp means percentage points of duty cycle). The same testbench was run seven times, each time with the model broken in one known way:

Injected fault	DC section	duty section	worst duty error
none	46 pass	14 pass	0.43 pp
comparator offset, +5 mV	1 fails	pass	0.43 pp
comparator offset, +20 mV	1 fails	pass	0.73 pp
comparator offset, +50 mV	2 fail	pass	0.77 pp
dynamics slowed 2×	pass	pass	0.65 pp
dynamics slowed 10×	pass	pass	0.87 pp
dynamics slowed 50×	pass	9 fail	14.3 pp

The table shows two things. First, the DC section is blind to *any* error in how fast the block responds. Slow the model down fiftyfold and all 46 of its DC checks still pass. Yet DC checks are all that many hand-written environments contain. The duty section exists to cover that blind spot, and it is the only check in the environment that catches the slowed model.

Second, the table shows where the test stops catching errors, and why. A tenfold slow-down shifts the output duty cycle by 0.87 pp. That is under the 1 pp the specification allows, so the test passes. The tool's own default pass/fail limit (bar) for this environment is 0.5 pp, which would have failed it.

An environment built from a specification is exactly as strict as the specification, and no stricter. This is deliberate. A requirement is what the circuit is held to; an environment that silently applied a tighter limit would report failures the design does not actually have. What the fault sweep adds is the number. The reader now knows that this specification cannot detect

a tenfold error in the block's dynamics, and can decide whether that is the requirement they meant to write.

The test is checked in both directions. A test that fails everything is as useless as one that passes everything. So the same sweep also confirms that small, acceptable errors do *not* trip the environment. The unbroken model passes, and a 5 mV offset shifts the duty cycle by nothing the duty section should object to.

The sweep is re-run against the shipped model whenever the environment changes, so the table always reflects the current environment. That is why this table differs from the earlier one measured against the default bar. The claims stay measured results rather than drifting into unchecked folklore.

10 Functional specifications as input

A netlist says what a circuit *is*. It does not say what the circuit is *for*, what rate it must run at, or how much error is acceptable. You can give the tool those facts as a plain-language document, such as a datasheet excerpt or a requirements page. The tool reads out the facts that shape verification:

From the specification	Used for
operating clock range	where the environment drives the block
accepted input duty range	the stimulus sweep
duty accuracy requirement	the pass/fail tolerance
block function	which environment is generated

Case study 2 is run this way. It reads `specs/dcc_spec.md` from the repository. Its duty-cycle environment checks 35–65% input duty at the specified clock rates, against the specification's own 1.0 pp requirement. Result: 14 checks, 0 failed, worst 0.428 pp. PASS

The specification is cross-checked, never trusted. Every fact taken from it is compared with what the circuit was actually measured to do, and the measurement wins. When the two disagree, the tool reports the disagreement as a finding:

```
SPEC CONFLICT: the spec says this block operates up to 4e+08 Hz, but
the fitted model is only valid below 1.195e+08 Hz -- 3.3x lower.
```

This is a coverage gap: the requirement asks for more than the characterisation covers. It is caught when the model is generated, not later at silicon correlation. Manual model review cannot find it, because no single document contains both the requirement and the measurement.

11 Joining an existing model library

A team that already has real-number models also has conventions: which user-defined nettype carries analog values, which package declares that nettype, and what port discipline the models use. Generated files that import a nettype the tool invented do not join that library. They sit beside it, the two net schemes cannot connect, and the customer pays the integration cost.

Point `--style-from` at the library and the output follows its conventions. The environment's analog nets use the library's nettype, its files import the library's package, the tool generates no types package of its own, and the generated run script compiles the library's package file. If the library's models use `wreal` ports, the generated model gets wreal ports automatically. The library's timescale is reported but deliberately not adopted. Generated delays are computed femtosecond values, and timescale applies per compilation unit, so the two timescale directives work together unchanged.

Reading a house style takes judgement, so the tool reads it and then verifies it. A library may declare several nettypes, of which only one is the current standard, and often only a comment says which. We measured this on a library that declares two nettypes, where the deprecated one happens to sort first alphabetically:

mechanical scan	picks the deprecated nettype, and warns
language-model reading	picks the current one, quoting the deprecation notice
verification	re-checks the choice against the named file

The language model's stated grounds, verbatim from that run: *"line 15 marks `aaa_legacy_wire` deprecated ("kept only so ancient testbenches still compile. Do NOT use in new code"), ... both models declare all analog ports as `acme_ewire`, confirming this is the live convention."* None of this is taken on trust. The nettype, package and file it names are checked against the file before use. If the file does not support the claim, the mechanical scan's answer stands, and the run states that it fell back. Where the tool has a *measurement* (here, a count of which port discipline the library's models actually use), the count wins outright. A disagreement with it is recorded, not adopted.

The result is an environment that runs on the library's own net type, and the generated files are published so you can inspect them. The showcase carries the same filter twice: `lpf2/` generated normally, and `lpf2_house/` generated against `house_lib/`, a small hand-written library standing in for a customer's own. Comparing the two directories with diff shows the whole effect:

	`lpf2/`	`lpf2_house/`
types package	generated	**not generated**
nets	`rc_lpf2_wire`	`ng_anet`
imports	the generated package	`ng_ams_pkg`
run script	compiles the generated package	compiles the house package
model ports	plain `real`	`ng_anet`, and the environment instantiates it
the model	byte-identical	
the scoreboard	byte-identical	

Following the house style changes only the boundary of the model. The published environment runs where it sits, against the library beside it: 49 DC and 15 AC checks, 0 failed, exactly the same result as the default environment.

Note what the environment instantiates in the second column. When the model is delivered inside a port wrapper, the generated testbench drives that wrapper module, not the core inside it, and the wrapper's file is added to the compile list. So the verification evidence covers what ships, not just one component of it. In separate measurements, a library whose nettype resolves multiple drivers by averaging rather than summing also conforms and passes the same way. A library using `wreal` ports gets wreal ports on the model instead. Without `--style-from`, or against a directory with nothing recognisable in it, nothing changes, and the run says so.

12 AI where it helps, physics where it decides

spice2rnm asks a language model four kinds of question: where one block ends and the next begins, what a block is for, what a written specification requires, and what conventions an existing model library follows (Section 11). Each of these is a question of *intent*. None is a question of physics.

The tool's rule is: **intent from the language model, physics from the measurement**. Every suggestion is checked against the measured circuit before it is used. If it does not match, it is discarded, the run continues with the measured data, and the run reports that it did so. With no model reachable at all, every AI feature switches off cleanly and the pipeline runs entirely on measurement.

Which of the two happened is recorded in the run's result, not only in its log. Each pass/fail verdict records the metric used to reach it. A reader of the report months later can therefore tell a check that ran as requested from one where the AI feature declined and the tool fell back to measurement. Without that record, every passing result would have to be trusted to have used the feature that was requested.

13 Scope and requirements

- **Inputs**: SPICE netlists, flat or with subcircuits. Optionally, a plain-language functional specification, and an existing RNM library whose conventions the generated models should follow.
- **Outputs**: SystemVerilog real-number models, using the port type of your choice: `wreal` by default, plain `real` where a flow has no wreal support, or your library's own nettype. Also UVM-MS 1.0 environments, and equivalence reports with archived waveforms.
- **Toolchain**: a SPICE simulator for characterisation, and a digital simulator that can run real-number models (with UVM support, for the generated environments). The generated models and environments are standard SystemVerilog with no vendor dependence, so either simulator can be the one your flow already uses.
- **Block-level scope**: single-input single-output blocks and compositions of them, including current-mode blocks that share a node. Composing blocks hierarchically gives up some accuracy in exchange for structure. The equivalence report states the composed figure separately from the per-block figures, so the accuracy cost is visible.
- **Jitter transfer is verified; the oscillator's own device noise is characterised.** Both are measured, but in different ways.

  Jitter transfer is what the model does with a disturbance it is given, and it is checked with a pass/fail limit. The disturbance is injected into the transistor-level circuit in SPICE, the same disturbance is replayed into the model, and the period jitter of the two is compared. This is done on both the control input and the supply. Section 8.3 gives the measurements: six points checked over a tenfold range, with a worst ratio of 0.940 against a 0.25 tolerance. Nobody asked for this check: the pipeline ran it on its own because the netlist declares noise on a net the ring oscillator is wired to. It is a real check with a pass/fail result, and it fails a model whose transfer is wrong.

  The oscillator's own noise is characterised through its impulse sensitivity function (ISF), $\Gamma(\phi)$: how much a small kick of charge at phase ϕ of the cycle permanently shifts the oscillator's phase. The pipeline measures it without being asked. It injects a narrow charge pulse at each phase of the cycle and measures the lasting phase shift; $\Gamma(\phi)$ is the sensitivity that results.[2] On the reference ring oscillator, the rms (root mean square) value Γ_{rms} is

[2] The method is Hajimiri, Limotyrakis and Lee, *Jitter and Phase Noise in Ring Oscillators*, IEEE JSSC 34(6), June 1999. Two details decide whether it works. Getting either wrong gives a plausible but wrong answer rather than an error, so the tool enforces both. First, the pulse is applied differentially where the circuit is differential. Injected into one node of a differential ring, it is mostly common mode, which the ring rejects by design; the measurement then reads its own noise floor however large the pulse. Second, linearity is checked at the phase of greatest sensitivity, not at a convenient one. The same charge that moves the phase by 0.04 ps where the ring barely responds moves it by 52 ps at the peak.

4.04×10^{12} rad/C from the automated sweep, against 4.33×10^{12} measured by hand: the two agree to 7%. Γ changes sign across the cycle, as a ring oscillator's must, and halves when the injected charge halves.

Multiplying Γ_{rms} by a device noise density gives a jitter: 0.236 to 0.578 ps rms here, or −114.5 to −106.7 dBc/Hz at 1 MHz. It is given as a range because two inputs are themselves ranges: the transistor channel-noise factor, and the way the noise densities of the individual nodes combine. The model carries this jitter as a parameter, shipped set to zero. The figure is computed from the measured ISF, not compared against a SPICE transient, because ngspice does not simulate device noise in a transient run.

Driven blocks jitter differently, and the models reflect this. An oscillator times each edge from its own previous edge. Its phase therefore does a random walk, and the timing error accumulates. A buffer or a lag block times its output from an *input* event. Each pass through the block gets an independent random delay, and nothing accumulates.[3] On that basis, the scheduled-lag models accept a synchronous-jitter figure and perturb the input-to-output delay once per pass through the block. This figure is supplied by the user, not measured: a SPICE transient of a netlist that declares no noise contains no delay variation to measure. The generated file says so. It also states the fixed extra latency the perturbation costs. The model's delay line can only look back in time, so an edge can be made early only relative to a delay that is already there.

Edge jitter is measured wherever the transistor-level circuit in SPICE has clock edges. On the reference PLL's two clock-path buffers it is 3.98 ps and 3.60 ps. It is listed next to the skew because jitter and skew go wrong in different ways. Edge jitter is shown for information, with no pass/fail limit, because there is no agreed threshold for it. Inventing one from a single design would be worse than showing the number.

The system-level equivalence check also shows its jitter with no pass/fail limit, and here the reason is the measurement, not the model. The check's boundary log is sampled every 100 ps. The composed model's output crosses its threshold between samples, so a crossing can only be located to the nearest sample. Reporting that as jitter would measure the sampling grid, not the circuit.

The generated UVM-MS environment has a monitor without that limitation. It reacts to each change of the signal instead of sampling on a clock. It is compiled with a `1fs` timescale while the rest of the environment stays at `1ps`; this costs nothing elsewhere, because SystemVerilog sets timescale per module. Running that environment on the composed PLL gives 12.70 ps rms period jitter over 1151 periods at the boundary. Over 116,012 boundary crossings, none of its system checks failed and one was inconclusive. The total number of checks is not quoted because it varies: one check exists only when the transistor-level circuit in SPICE settled measurably, which under a noisy supply it may or may not do.

This 12.88 ps figure is also shown for information, with no pass/fail limit, for a specific reason. A PLL suppresses its oscillator's jitter inside the loop bandwidth. A figure measured at the PLL boundary is therefore not the oscillator's own jitter, and the two are not expected to

[3] The distinction and both implementations are Kundert's, in *Modeling and Simulation of Jitter in Phase-Locked Loops*, AACD 1997. This model follows four of his points. Driven systems jitter without accumulating; autonomous ones (free-running oscillators) have a phase difference that is "a random walk that is not bounded". The metric for both is $J = \sigma(T)$, the standard deviation of one period. Driven jitter is modelled by "randomly dithering the delay argument ... on every output transition". And with N updates per period, the per-update dither scales by $\sqrt{N}$, which is 1.414 for the two updates this model uses.

Each point was checked. Asked for 1% rms period jitter, the model produced 0.9998%. Its scaling constant is the $\sqrt{2}$ his listing specifies. Over 30,000 cycles, the variance of the accumulated deviation grew linearly with cycle count, at 1.004 ± 0.026 of the kJ^2 his random walk requires.

One setting is refused. At one sub-step per half cycle the scheduler cannot resolve a randomised threshold, so the model rejects that setting rather than report 39.6% jitter when 1% was asked for.

agree. The 10.72 ps that this netlist's declared supply noise produces at `ro_vco` is a different measurement of a different quantity. On this run the two differ by 20%; on the previous run, by 16%. Each is one run's result from a single random sample, and neither is evidence about the other.

This validation goes further than published practice. The gap named above (no intrinsic device noise in the model) is one that published practice shares. Analog Devices' production RNM suite for a GHz transceiver validates its PLL on lock time and frequency error, the same quantities checked in Section 8. It states that "model validation remains an area for improvement in the real number modeling flow", and names frequency-domain performance as future work.[4]

A What the routes measure

A route is the characterisation method the tool chooses for a block. The case studies name each block's route and say why the block needs it. This appendix explains what the routes actually measure, using the reference PLL's own results rather than an invented example. You do not need it to read the results; it is here for a reader who wants to check them.

A.1 Why an oscillator is compared as a rate

Most blocks are checked by lining up two output waveforms in time and subtracting them. That is the first row of the table in Section 8. An oscillator, the fourth row, cannot be checked this way, and the reason is the whole justification for its route.

Suppose the model and the transistor-level circuit in SPICE run at *exactly* the same frequency: a perfect model. They start aligned. But nothing locks their phases together. A frequency difference of one part in 10^5 slides them a full cycle apart within a millisecond. From then on, the model sits at one supply rail whenever the circuit sits at the other. Subtracting the two waveforms gives full-swing error everywhere, and the perfect model scores as entirely wrong.

This is not hypothetical. A milder version appears elsewhere in this document: the two clock buffers in case study 4 report a worst instantaneous error of 99.9% of full swing, because an edge lands slightly late. For a buffer, that error lasts a brief moment each cycle. For an oscillator, it would be the permanent steady state.

So the oscillator route subtracts no waveforms. Instead it compares frequencies. It sets the control voltage, runs the circuit in SPICE and counts cycles, runs the model at the same control voltage and counts its cycles, then compares the two frequencies:

set `cont` to	1.9000 V
SPICE: count cycles, divide	f_{spice}
model: same control, count	f_{model}
score	$\lvert f_{\text{model}} - f_{\text{spice}} \rvert / f_{\text{spice}} = 0.024\%$
pass/fail limit	2%

Phase is ignored on purpose. The model does not claim to reproduce it: the model generates an ideal square wave. The real ring oscillator's finite edge rates were already taken into account when its frequency was measured by FFT.

Every verdict records which of these rows it came from. So months later, a reader can still tell which number decided the result and which numbers were only noted alongside it.

[4] J. Dudek, J. Nekl and K. O'Donoghue (Analog Devices), *Real Number Modeling of RF Circuits*, DVCon proceedings, `https://dvcon-proceedings.org/wp-content/uploads/real-number-modeling-of-rf-circuits.pdf`. The paper notes that because RNMs exist to enable system-level testing, "the requirements for model validation are greatly relaxed in this environment".

A.2 The tuning curve

For an oscillator, the tuning curve is not a description of the model: it *is* the model. The oscillator route measures a table with one row per control voltage. Each row is a separate transient simulation of the transistor-level circuit in SPICE, with the control voltage held fixed and the resulting oscillation timed:

`cont` (V)	frequency (MHz)
0.495	804.47
⋮	⋮
1.750	438.73
1.850	399.86
⋮	⋮
2.805	16.74

The generated model looks up its frequency in this table and interpolates between rows. That is why its checks come in three kinds:

- **At the table's own 15 rows.** Here the model is exact by construction. Agreement (0.001%) only shows that the table was copied correctly.
- **At 5 control voltages *deliberately withheld* from the table.** Here the model must interpolate, and it is compared against fresh SPICE measurements. The result, 0.024%, is the only one of the three that is evidence about accuracy.
- **At 9 voltages far outside the closed loop's operating range.** Here the error reaches 4.987%. This is shown for information, with no pass/fail limit, because the loop never goes there.

Section 6.4 reads the same table backwards. For 400 MHz it gives 1.8496 V, and the assembled loop is then required to settle at that control voltage.

A.3 Control states

A control state is one combination of the digital gate inputs that drive a block. The charge pump has two gate inputs, `pdn` and `pupb`, driven *independently*. So it has four states rather than one continuous control input. Each state has its own measured current-versus-output-voltage table, $I(V)$: 12 points from 0 to 3.3 V, with 11 more held out for checking.

state	(`pdn`,`pupb`)	i at 1.80 V	at 3.3 V	what it does
00	(0, 0)	+7.239 mA	≈ 0	sources: charges the filter
01	(0, 1)	−1.9 nA	−4.0 nA	high-Z: off
10	(1, 0)	+2.798 mA	−5.078 mA	both on, fighting
11	(1, 1)	−4.441 mA	−5.078 mA	sinks: discharges it

Why one table per state, not one fitted curve. The gates are digital: each is on or off. So the block has no continuous control input to fit a curve against. Instead it has a small, fixed set of modes. Within a mode, its current depends only on the output voltage, because how much current it can push into a node depends on the voltage already there. That is what $I(V)$ means here, and why there are four tables.

This is also why the default route cannot model the pump. The pipeline decides this from the circuit's topology, not from the block's name. An inverter ties its two gates together, so

one input drives both and there is a single input to fit against. In the pump, the two gates are separate digital signals. That makes states 01 and 10 reachable, and an inverter has neither. State 01 is the idle state, drawing nanoamps; state 10 has both transistors conducting at once.

These four rows matter in two more places in this document. First, the loop harness of Section 6.4 uses them to work out each gate's active level. It reads them at 1.849 V, where the loop settles, rather than at a table row. With `pupb` idle, `pdn` at 1 reaches state 11 and sinks 4.47 mA, so `pdn` is active high. With `pdn` idle, `pupb` at 0 reaches 00 and sources 7.103 mA, so `pupb` is active low. Second, the equivalence verdict names the state where its worst error occurred: 0.5487% against a 2% pass/fail limit, at state 00. An average across the four modes would hide which mode is weakest.

A.4 The state space

Because a filter can be written exactly as a state space, its route needs to measure nothing at all. A state space is the standard way to write a linear dynamic system: a few variables whose *rates of change* depend on their present values and on the input, $\dot{x} = Ax + Bu$, $y = Mx + Nu$. For an RC network, the state variables are just the capacitor voltages. The netlist is:

```
.subckt lpfilt dra vout
R1 dra  int1 300        C1 c1   0  10n
R2 int1 c1   200        C2 int1 0   5n
R3 int1 vout 50         C3 vout 0  0.5n
.ends
```

Three capacitors give three states: the voltages on `c1`, `int1` and `vout`. The input is the current the charge pump injects into `dra`; the output is `vout`. When the paper says this route is "exact and needs no fitting", it means every coefficient in the generated model is computed from values on those six lines:

generated coefficient	is	which is
500,000	$1/(R_2C_1) = 1/(200 \times 10\,\text{n})$	the `c1` pole
40,000,000	$1/(R_3C_3) = 1/(50 \times 0.5\,\text{n})$	the `vout` pole
1,000,000	$1/(R_2C_2) = 1/(200 \times 5\,\text{n})$	`c1` into `int1`
4,000,000	$1/(R_3C_2) = 1/(50 \times 5\,\text{n})$	`vout` into `int1`
200,000,000	$1/C_2 = 1/5\,\text{n}$	the injected current

None of these came from a simulation. Compare the inverter in the same netlist: there the tool ran SPICE 33 times, fitted a curve, and reports an rms (root mean square) fit error. Here there is nothing to fit and nothing to get wrong, because Kirchhoff's laws and the netlist fully determine the answer. The equivalence check can still catch a code-generation error, meaning the generated file disagrees with the matrices it was built from. That is what its 0.019% measures.

Two details of the generated file follow from the same reasoning. First, `dra` is *not* a state, because it has no capacitance. Its voltage is set at each instant by the resistors and the injected current, so it is solved algebraically at each time step, together with the pump that drives it. Second, the filter has no DC path to ground. A constant injected current therefore ramps the output forever without settling. That is exactly what a charge pump driving a capacitive filter does, and it is why the filter is modelled as a state space rather than a DC gain.

© 2026 Veylon Systems. spice2rnm is a Veylon Systems product. Every figure in this document was produced by the shipped tool from its bundled demonstration circuits, most recently on 2026-09-22. The demonstration suite at `https://github.com/vrajeshprakhya/spice2rnm-demos` regenerates them in one command.